SOROCABA, SP

2026

EDUARDO DE SOUZA MONTEIRO 212123

FELLIPE ALESSANDRO SCRUPH 222378

GUILHERME MATHEUS FONSECA WAIZBART 211899

JOÃO VICTOR DE OLIVEIRA 212106

RENAN VIZOTO FERREIRA 222220

VINICIUS CÂNDIDO DAS CHAGAS 211873

MODELAMENTO DO CONTROLE EM MALHA FECHADA DA MISTURA AR-COMBUSTÍVEL AUTOMOTIVA

Documento de Projeto RTOS para a disciplina TEECII.

Orientadores: Adilson Rocha e Marcos Fábio Jardini.

RESUMO

Este trabalho apresenta o desenvolvimento de uma bancada simulada para o controle em malha fechada da mistura ar-combustível de um motor de combustão interna, a partir da leitura de uma sonda lambda. A unidade de controle eletrônico (ECU) é um firmware em C, com HAL e FreeRTOS, para o microcontrolador STM32F072, executado sem hardware físico no emulador Renode. O motor, o atraso de transporte dos gases no escapamento e a dinâmica da sonda são modelados por uma planta escrita em Go, na qual cada sensor é uma rotina concorrente, sincronizada com o emulador a cada passo de simulação. A ECU lê os sensores por conversor analógico-digital com DMA, calcula o tempo de injeção por uma tabela de alimentação com correção proporcional-integral e gera o pulso do injetor por temporizador; o estado do sistema é publicado em barramento CAN e exibido em um painel gráfico nativo. Nos ensaios, a tarefa de controle cumpriu o período de 10 ms com jitter máximo de 10 µs, tempo de execução de cerca de 160 µs e nenhum prazo perdido, e a mistura retornou a $\lambda = 1{,}00 \pm 0{,}02$ em 1,3 s, sem sobressinal, após uma redução de 20 % na vazão do injetor. A bancada permite estudar, de forma segura e reprodutível, o controle em malha fechada sob restrições de tempo real.

Palavras-chave: Sistemas de tempo real. FreeRTOS. Sonda lambda. Controle PI. STM32.

ABSTRACT

This work presents a simulated test bench for closed-loop control of the air-fuel mixture of an internal combustion engine based on a lambda sensor reading. The engine control unit (ECU) is C firmware, built on the vendor HAL and FreeRTOS, for the STM32F072 microcontroller, running without physical hardware on the Renode emulator. The engine, the exhaust gas transport delay and the sensor dynamics are modeled by a plant written in Go, in which each sensor is a concurrent routine synchronized with the emulator at every simulation step. The ECU samples the sensors through an analog-to-digital converter with DMA, computes the injection time from a fuel map with proportional-integral correction and drives the injector pulse with a hardware timer; the system state is published on a CAN bus and shown on a native graphical panel. In the tests, the control task met its 10 ms period with a maximum jitter of 10 µs, an execution time of about 160 µs and no missed deadlines, and the mixture returned to $\lambda = 1.00 \pm 0.02$ within 1.3 s, without overshoot, after a 20 % reduction in injector flow. The bench provides a safe and reproducible platform for studying closed-loop control under real-time constraints.

Keywords: Real-time systems. FreeRTOS. Lambda sensor. PI control. STM32.

LISTA DE FIGURAS

Figura 1 - Arquitetura da bancada simulada.....5
Figura 2 - Fluxograma da tarefa de controle (ECU).....6
Figura 3 - Fluxograma da planta simulada.....6
Figura 4 - Painel do operador.....10
Figura 5 - Resposta a uma redução de 20 % na vazão do bico (2000 rpm, malha fechada).11

LISTA DE TABELAS

Tabela 1 - Cronograma de desenvolvimento.....2
Tabela 2 - Janelas de tempo e metas por etapa.....7
Tabela 3 - Metas de tempo real e valores medidos.....8
Tabela 4 - Tarefas, interrupções e mecanismos de sincronização.....8

SUMÁRIO

1 INTRODUÇÃO....1
1.1 INTRODUÇÃO DO GRUPO....1
1.2 DESCRIÇÃO DA ÁREA DO TEMA, RELEVÂNCIA E PONTOS CRÍTICOS....1
1.2.1 Descrição dos objetivos específicos e escopo....1
1.3 CRONOGRAMA GERAL DE DESENVOLVIMENTO (MACRO TAREFAS)....2
2 RESUMO DE SISTEMAS OPERACIONAIS....2
2.1 RESUMO DE IMPLEMENTAÇÃO DE UM RTOS: FREERTOS....3
3 DESCRIÇÃO DO PROCESSO MODELADO....3
3.1 CÁLCULOS DE TEMPOS E GARANTIAS DE ATUAÇÃO....6
4 DESENVOLVIMENTO....8
4.1 TAREFAS E RECURSOS DO FREERTOS....8
4.2 TRECHOS DO CÓDIGO-FONTE....9
4.3 INTERFACE, TESTES E PRIMEIROS RESULTADOS....10
5 CONSIDERAÇÕES FINAIS....11
REFERÊNCIAS....12

1 INTRODUÇÃO

1.1 INTRODUÇÃO DO GRUPO

O presente trabalho foi desenvolvido pelos alunos Eduardo de Souza Monteiro, Fellipe Alessandro Scruph, Guilherme Matheus Fonseca Waizbart, João Victor de Oliveira, Renan Vizoto Ferreira e Vinicius Cândido das Chagas, estudantes do curso de Engenharia de Computação da Facens. O projeto faz parte das atividades práticas da disciplina de Tópicos Especiais de Engenharia, orientada pelos professores Adilson Rocha e Marcos Fábio Jardini.

1.2 DESCRIÇÃO DA ÁREA DO TEMA, RELEVÂNCIA E PONTOS CRÍTICOS

O tema escolhido aborda sistemas embarcados aplicados à área automotiva, com foco no controle eletrônico da injeção de combustível. Manter a proporção correta entre ar e combustível, indicada pelo fator lambda medido pela sonda, é essencial para que o veículo polua menos, economize combustível e funcione de maneira estável. Na prática, esse controle enfrenta desafios importantes, como a diferença de resposta entre sensores comuns e sensores de precisão e o tempo que os gases da queima levam para sair dos cilindros e alcançar o sensor no escapamento. Esse atraso varia com a rotação do motor e pode desestabilizar o controle se o sistema não for rápido e bem ajustado.

1.2.1 Descrição dos objetivos específicos e escopo

O objetivo do grupo é construir uma bancada de testes simulada para monitorar a sonda lambda e ajustar a mistura em malha fechada sem um motor real. O firmware da ECU é escrito para o STM32 e executado no emulador Renode; o modelo do motor, o atraso do escapamento (buffer circular) e a resposta da sonda (equação de primeira ordem) ficam em uma planta simulada externa, que se comunica com o microcontrolador emulado pelas entradas analógicas e pela saída do injetor. Os cálculos de tempo concentram-se na taxa de amostragem do conversor analógico, no período e no jitter da tarefa de controle e na largura do pulso de injeção em microssegundos. Com esses dados, analisa-se quanto tempo o sistema leva para corrigir a mistura após uma perturbação e se o controle permanece estável.

1.3 CRONOGRAMA GERAL DE DESENVOLVIMENTO (MACRO TAREFAS)

Tabela 1 - Cronograma de desenvolvimento

Etapa	Macro Tarefa	Descrição / Entregável
1	Levantamento Teórico e Equações	Definição das fórmulas da mistura, do atraso do escapamento e da resposta da sonda lambda.
2	Configuração dos Periféricos do STM32	Configuração, no emulador Renode, dos timers de disparo, da leitura analógica por DMA, do pulso do injetor e do CAN.
3	Programação da Planta Simulada	Implementação, em Go, do modelo do motor, do atraso do escapamento e da sonda, sincronizado com o microcontrolador emulado.
4	Implementação do Código de Controle	Desenvolvimento da lógica de malha fechada para calcular a correção do tempo de injeção.
5	Testes de Resposta e Coleta de Tempos	Medição do tempo de resposta da malha fechada, avaliação de atrasos e validação do sistema na bancada.
6	Escrita e Entrega do Relatório	Organização dos gráficos gerados, análise das medições de tempo e elaboração do documento final.

2 RESUMO DE SISTEMAS OPERACIONAIS

Um sistema operacional (SO) é a camada de software situada entre o hardware e as aplicações. Ele cumpre dois papéis: oferecer abstrações mais simples que o hardware (arquivos, processos, dispositivos) e gerenciar os recursos do computador, como processador, memória, tempo e dispositivos de entrada e saída, entre programas que competem por eles (TANENBAUM; BOS, 2016).

A unidade de execução gerenciada pelo SO é o processo, isto é, um programa em execução com seu próprio espaço de endereçamento, registradores e estado. Um processo pode conter várias threads, que compartilham a memória e são escalonadas de forma independente. O escalonador decide qual tarefa ocupa o processador, alternando-as entre os estados pronto, executando e bloqueado. Como tarefas concorrentes acessam dados comuns, o SO fornece mecanismos de sincronização e comunicação, como semáforos, mutexes e filas de mensagens, que protegem as seções críticas e evitam condições de corrida.

O acesso aos dispositivos é feito por drivers, módulos que encapsulam os registradores e as interrupções de cada periférico e expõem uma interface uniforme ao restante do sistema. Em microcontroladores, esse papel costuma ser desempenhado por bibliotecas do fabricante, como a HAL (Hardware Abstraction Layer) da STMicroelectronics usada neste projeto.

Um sistema operacional de tempo real (RTOS) é aquele em que a correção depende não só do resultado, mas também do instante em que ele é produzido (KOPETZ, 2011). Os prazos podem ser rígidos (hard), quando perdê-los causa falha; firmes (firm), quando o resultado atrasado perde o valor; ou flexíveis (soft), quando apenas degradam o desempenho. Por isso, um RTOS privilegia o determinismo em vez da vazão média: escalonamento preemptivo por prioridade fixa, latência de interrupção limitada, serviços de tempo de execução previsível e pequeno consumo de memória. A análise clássica de escalonabilidade de Liu e Layland (1973) mostra que n tarefas periódicas com prioridades atribuídas pela taxa (menor período, maior prioridade) cumprem seus prazos se a utilização total do processador não exceder $n(2^{1/n} - 1)$.

2.1 RESUMO DE IMPLEMENTAÇÃO DE UM RTOS: FREERTOS

O FreeRTOS é um núcleo de tempo real de código aberto (licença MIT), portado para mais de quarenta arquiteturas e com poucos kilobytes de código (BARRY, 2016). A aplicação é dividida em tarefas, cada uma com pilha e prioridade próprias, nos estados executando, pronta, bloqueada ou suspensa. O escalonador é preemptivo: a tarefa pronta de maior prioridade sempre executa, e tarefas de mesma prioridade revezam a cada tick. O tick é uma interrupção periódica que serve de base para atrasos e temporizações, como a função vTaskDelayUntil, que mantém um período fixo sem acumular erro.

Para comunicação e sincronização, o FreeRTOS oferece filas (cópia de dados por valor, seguras entre tarefas e interrupções), semáforos binários e contadores, mutexes com herança de prioridade e notificações diretas de tarefa, uma alternativa mais leve ao semáforo que não exige objeto adicional. Rotinas de interrupção usam as variantes FromISR das funções e solicitam troca de contexto com portYIELD_FROM_ISR. Trechos curtos que não podem ser interrompidos são protegidos por taskENTER_CRITICAL e taskEXIT_CRITICAL. Neste projeto, o núcleo roda no Cortex-M0 a 48 MHz, com preempção habilitada, tick de 1 kHz, cinco níveis de prioridade e alocador heap_4 com 7 KB.

3 DESCRIÇÃO DO PROCESSO MODELADO

A qualidade da mistura é expressa pelo fator lambda, razão entre a relação ar-combustível real e a estequiométrica (AFR_{st} = 14,7 para a gasolina pura):

$$\lambda = m_{ar} / (AFR_{st} \cdot m_{comb})$$

Com $\lambda = 1$ a queima é completa e o catalisador de três vias opera com máxima eficiência; $\lambda < 1$ indica mistura rica e $\lambda > 1$, pobre (HEYWOOD, 2018; BOSCH, 2005). O processo modelado é a cadeia injetor → combustão → escapamento → sonda → ECU. O motor simulado tem quatro cilindros e 1,6 L. A massa de ar por ciclo segue o modelo alfa-N (borboleta e rotação), e a de combustível depende do tempo de abertura do bico t_i, descontado o tempo morto de abertura:

$$m_{ar} = m_{ar,max} \cdot f(TPS) \cdot \eta(RPM) + m_{vaz}$$

$$m_{comb} = k_{inj} \cdot Q_{inj} \cdot (t_i - t_{morto})$$

Nessas equações, $Q_{inj} = 3$ mg/ms, $t_{morto} = 0{,}8$ ms e $m_{ar,max} = 480$ mg. Os termos m_{vaz} (vazamento de ar não medido) e k_{inj} (bico sujo ou desgastado) são as perturbações aplicadas nos ensaios. O gás leva um tempo T_d para chegar à sonda, que responde como um sistema de primeira ordem com $\tau = 100$ ms:

$$T_d = 180 / RPM + 0{,}020 \text{ s} \qquad \tau \cdot d\lambda_s/dt = \lambda(t - T_d) - \lambda_s$$

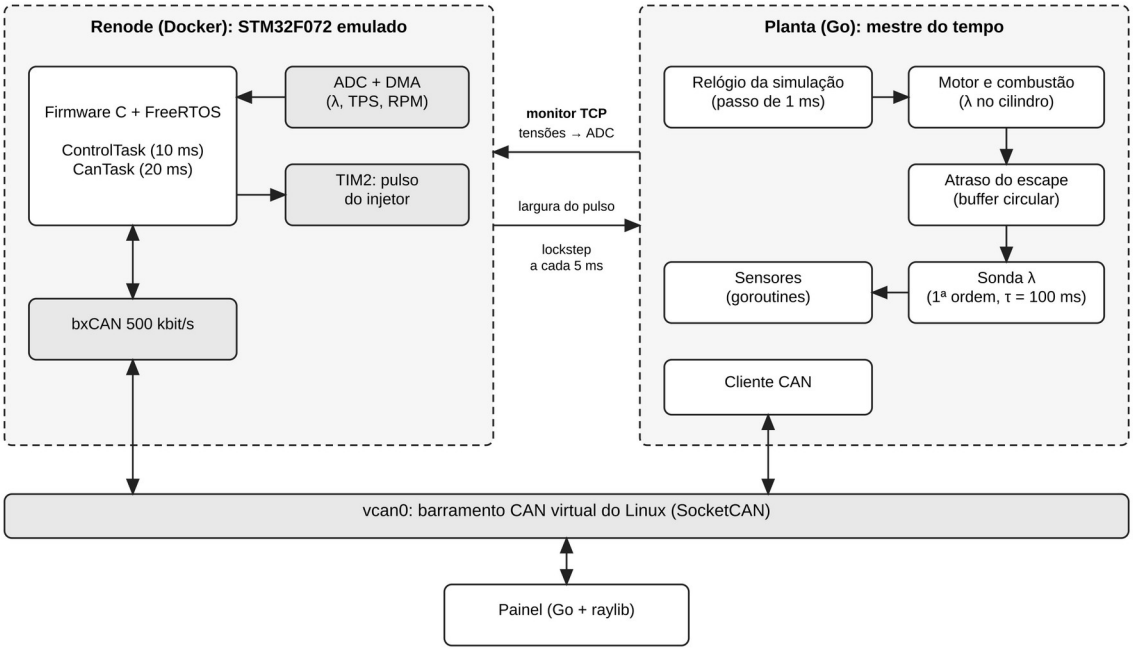
A ECU estima a parcela de combustível por uma tabela de alimentação $t_{base}(TPS, RPM)$ de 7 × 8 pontos, com interpolação bilinear, e corrige o erro residual com um controlador proporcional-integral (OGATA, 2010) sobre o erro $e = \lambda_{med} - \lambda_{alvo}$:

$$t_i = t_{morto} + t_{base} \cdot (1 + c), \qquad c = K_p \cdot e + K_i \cdot \int e \, dt$$

com $K_p = 0{,}15$, $K_i = 2{,}0$ s^{-1} e $|c| \le 0{,}30$. Um anti-windup condicional impede que o integrador cresça enquanto a saída estiver saturada. Em malha aberta, $c = 0$.

A Figura 1 mostra a arquitetura da bancada. O Renode emula o STM32F072 executando o mesmo binário que seria gravado na placa. A planta em Go é o mestre do tempo: a cada 5 ms de tempo simulado, escreve as tensões dos sensores nos canais do ADC, avança a emulação e lê a largura do pulso programada no temporizador do injetor, como faria uma ponta de osciloscópio. O controlador CAN emulado é ligado à interface virtual vcan0 do Linux, por onde o painel recebe o estado da ECU e da planta e envia comandos de acelerador, perturbação e modo de malha.

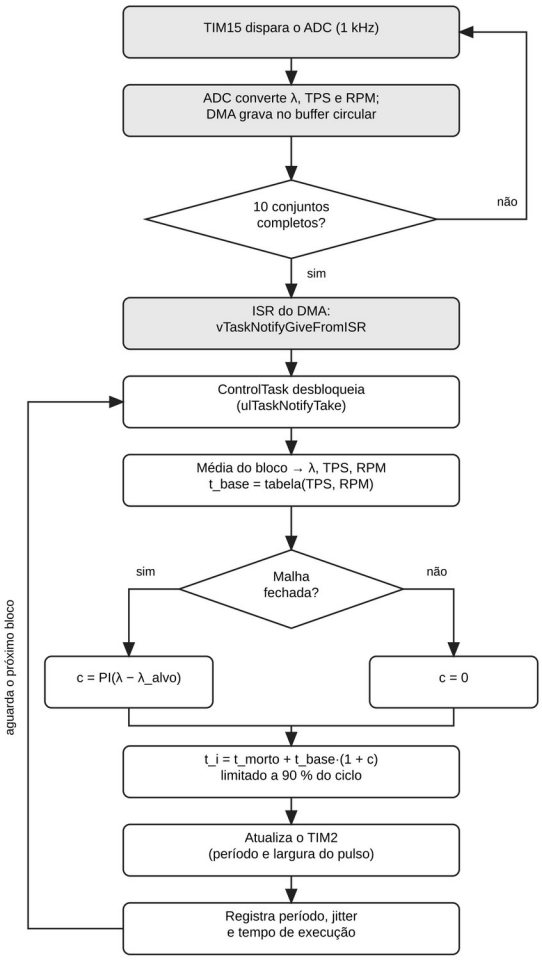
Figura 1 - Arquitetura da bancada simulada



Fonte: Autoria própria (2026).

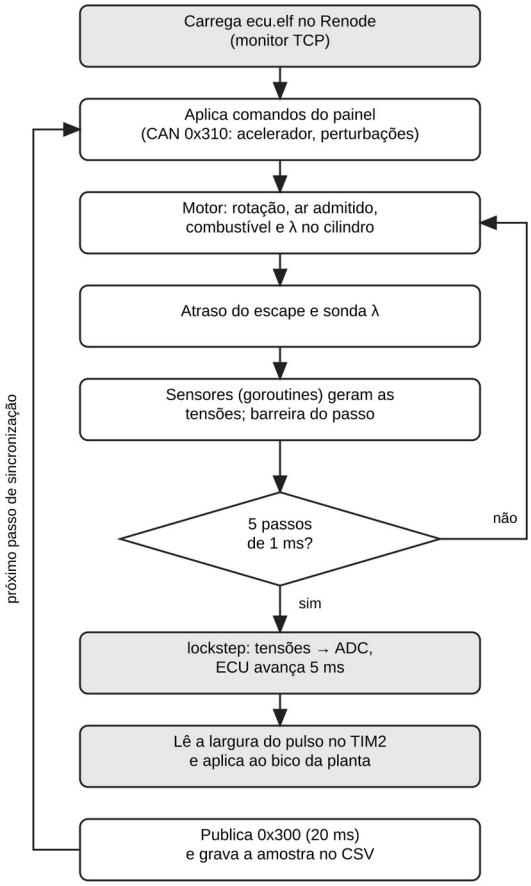
As Figuras 2 e 3 detalham os dois laços. Na ECU, a amostragem é feita por hardware (temporizador, ADC e DMA) e só o fim de cada bloco de 10 amostras acorda a tarefa de controle. Na planta, a física avança em passos de 1 ms e a sincronização com a ECU ocorre a cada cinco passos.

Figura 2 - Fluxograma da tarefa de controle (ECU)



Fonte: Autoria própria (2026).

Figura 3 - Fluxograma da planta simulada



Fonte: Autoria própria (2026).

3.1 CÁLCULOS DE TEMPOS E GARANTIAS DE ATUAÇÃO

A Tabela 2 reúne as janelas de tempo de cada etapa. A malha de lambda é lenta, dominada pelo atraso do escapamento (50 a 245 ms) e pela constante da sonda (100 ms), de modo que um período de controle de 10 ms amostra a planta de 5 a 25 vezes mais rápido que seu atraso. O prazo de cada ativação da tarefa de controle é o próprio período: o cálculo deve terminar antes que o DMA complete o bloco seguinte.

Tabela 2 - Janelas de tempo e metas por etapa

Etapa	Período / janela	Meta ou garantia	Classe
Amostragem (TIM15 → ADC → DMA)	1 ms por conjunto de 3 canais	Disparo por hardware, sem uso da CPU	Hard (hardware)
ControlTask: média, PI e TIM2	10 ms (bloco de 10 amostras)	Jitter ≤ 100 µs; execução ≤ 1 ms; nenhum prazo perdido	Firm
Pulso do injetor (TIM2, 1 µs)	1 ciclo do motor: 20 a 150 ms	Resolução de 1 µs; t_i ≤ 90 % do ciclo	Hard (hardware)
CanTask: estado, tempos e comandos	20 ms (tempos a cada 100 ms)	Sem prazo rígido	Soft
Malha de lambda	Atraso de 50 a 245 ms	λ = 1,00 ± 0,02 em ≤ 2 s; sobressinal ≤ 5 %	Soft

Fonte: Autoria própria (2026).

Utilização da CPU. Com execução medida de C ≈ 0,16 ms para T = 10 ms, a tarefa de controle ocupa U = C/T ≈ 1,6 % do processador. Mesmo supondo que a CanTask consuma 1 ms a cada 20 ms (5 %), a utilização total (≈ 6,6 %) fica muito abaixo do limite de Liu e Layland para duas tarefas, $2(\sqrt{2} - 1) \approx 82{,}8$ %. As prioridades seguem a mesma regra: a tarefa de menor período (controle, 10 ms) tem a maior prioridade.

Pulso de injeção. O motor de quatro tempos injeta uma vez a cada duas voltas, logo o ciclo vale $T_{ciclo} = 120/RPM$: 60 ms a 2000 rpm e 20 ms a 6000 rpm. A 2000 rpm e 23 % de borboleta, $t_i \approx 3{,}5$ ms (5,8 % do ciclo). No pior caso, a 6000 rpm, com borboleta toda aberta e correção de +30 %, $t_i \approx 12{,}2$ ms, ou 61 % do ciclo; o limite de 90 % é apenas uma proteção. O TIM2 conta a 1 MHz, então a resolução de 1 µs corresponde a menos de 0,04 % do combustível injetado, desprezível diante da faixa de ±2 % em λ.

Latência e barramento. O valor aplicado ao injetor é a média das amostras dos últimos 10 ms e é atualizado no TIM2 em até jitter + execução (≈ 0,17 ms) após o fim do bloco. A 500 kbit/s, um quadro CAN de 8 bytes ocupa cerca de 0,25 ms; com três a quatro quadros a cada 20 ms, a carga do barramento fica abaixo de 5 %.

Medição. A ECU registra, com um temporizador de 1 µs (TIM7), o instante de liberação de cada bloco e o início e o fim de cada ativação. Desses instantes saem o período, o jitter (desvio do período nominal), o tempo de execução e os prazos perdidos (ativações acumuladas ou término após o próximo bloco), publicados no quadro CAN 0x110. Os tempos são do relógio virtual do Renode, que não reproduz ciclo a ciclo o processador real. A Tabela 3 compara as metas com os valores medidos.

Tabela 3 - Metas de tempo real e valores medidos

Métrica	Meta	Medido	Ensaio
Período da ControlTask	10 ms	10 000 µs	Renode e ponta a ponta
Jitter da ControlTask	≤ 100 µs	1 a 10 µs	Renode e ponta a ponta
Execução da ControlTask	≤ 1 ms	158 a 161 µs	Renode e ponta a ponta
Prazos perdidos	0	0	Renode e ponta a ponta
Resolução do pulso	1 µs	1 µs	Renode
Acomodação (bico −20 %, 2000 rpm)	≤ 2 s	1,30 s	Ponta a ponta
Sobressinal	≤ 5 %	0 %	Ponta a ponta

Fonte: Autoria própria (2026).

4 DESENVOLVIMENTO

4.1 TAREFAS E RECURSOS DO FREERTOS

O firmware tem duas tarefas e duas rotinas de interrupção relevantes (Tabela 4). A ControlTask é acordada por notificação de tarefa, usada como um semáforo contador leve: a interrupção do DMA incrementa o contador e a tarefa o consome com ulTaskNotifyTake. Se o valor lido for maior que 1, houve ativações acumuladas, que são contadas como prazos perdidos. A recepção CAN usa uma fila de oito quadros: a interrupção apenas copia o quadro para a fila, e a CanTask a esvazia em seu ciclo de 20 ms. Os dados trocados entre as tarefas ficam em uma estrutura compartilhada protegida por seção crítica. Como a cópia leva poucos microssegundos, mascarar as interrupções nesse trecho é mais simples e previsível que usar um mutex, que acrescentaria trocas de contexto.

Tabela 4 - Tarefas, interrupções e mecanismos de sincronização

Elemento	Prioridade	Ativação	Recurso do FreeRTOS	Função
ISR do DMA	Hardware	Fim de bloco (10 ms)	vTaskNotifyGiveFromISR	Acordar a ControlTask
ControlTask	3	Notificação	ulTaskNotifyTake; seção crítica	Média, tabela + PI, TIM2, medição de tempos
ISR de recepção CAN	Hardware	Quadro recebido	xQueueSendFromISR (fila de 8)	Entregar comandos à CanTask
CanTask	2	Periódica, 20 ms	xTaskDelayUntil; xQueueReceive; seção crítica	Publicar estado e tempos; tratar comandos; log serial
Idle	0	Tempo livre	-	Tarefa ociosa do núcleo

Fonte: Autoria própria (2026).

4.2 TRECHOS DO CÓDIGO-FONTE

O Código 1 mostra a passagem da interrupção do DMA para a tarefa de controle. O período da tarefa é imposto pelo hardware, e não por um atraso de software, o que elimina a deriva entre a amostragem e o cálculo.

Código 1 - Interrupção do DMA e laço da ControlTask (adc_dma.c e app.c)

```
/* adc_dma.c: fim de um bloco de 10 conjuntos de amostras */
BaseType_t woken = pdFALSE;
vTaskNotifyGiveFromISR(notify_task, &woken);
portYIELD_FROM_ISR(woken);

/* app.c: ControlTask, prioridade 3 */
for(;;) {
    uint32_t n = ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
    control_step(n > 1u ? n - 1u : 0u);   /* n > 1: ativação perdida */
}
```

Fonte: Autoria própria (2026).

O Código 2 mostra a fila de recepção CAN e o laço periódico da CanTask, cuja cadência fixa de 20 ms é garantida por xTaskDelayUntil.

Código 2 - Fila de recepção CAN e laço da CanTask (canbus.c e app.c)

```
/* canbus.c: interrupção de recepção do bxCAN */
frame.id = header.StdId;
frame.len = (uint8_t)header.DLC;
xQueueSendFromISR(rx_queue, &frame, &woken);

/* app.c: CanTask, prioridade 2 */
for(;;) {
    while(canbus_receive(&rx, 0)) handle_command(&rx);  /* esvazia a fila */
    taskENTER_CRITICAL(); s = shared; taskEXIT_CRITICAL();
    /* ... envia 0x100 e 0x101; a cada 100 ms, 0x110 (tempos) ... */
    xTaskDelayUntil(&wake, pdMS_TO_TICKS(CAN_PERIOD_MS));
}
```

Fonte: Autoria própria (2026).

O Código 3 traz o núcleo do controlador PI com anti-windup condicional: o integrador só avança enquanto a saída não ultrapassa o limite de ±30 %.

Código 3 - Passo do controlador PI (controller.c)

```
static float pi_step(controller_t *c, float erro)
{
    float p = PI_KP * erro;
    float i = c->integral + PI_KI * erro * CONTROL_DT_S;
    if(erro > 0.0f && p + i > PI_LIMITE)
        i = maxf(c->integral, PI_LIMITE - p);
    else if(erro < 0.0f && p + i < -PI_LIMITE)
        i = minf(c->integral, -PI_LIMITE - p);
    c->integral = clampf(i, -PI_LIMITE, PI_LIMITE);
    return clampf(p + c->integral, -PI_LIMITE, PI_LIMITE);
}
```

Fonte: Autoria própria (2026).

Na planta, cada sensor é uma goroutine (thread leve do Go). A cada passo, o estado do motor é enviado por canal a todos os sensores, e uma barreira (sync.WaitGroup)

espera que todos respondam antes de prosseguir (Código 4), de modo que as leituras ficam coerentes no tempo simulado.

Código 4 - Barreira de sincronização dos sensores (sensors/bank.go)

```
func (b *Bank) Step(dt float64, st engine.State) []float64 {
    b.wg.Add(len(b.sensors))
    for _, c := range b.in {
        c <- tick{dt: dt, st: st}   // acorda cada goroutine de sensor
    }
    b.wg.Wait()                     // barreira: todos leram o passo
    return append([]float64(nil), b.out...)
}
```

Fonte: Autoria própria (2026).

4.3 INTERFACE, TESTES E PRIMEIROS RESULTADOS

O painel (Figura 4), escrito em Go com a biblioteca gráfica raylib, ocupa uma única tela e lê o barramento CAN. Ele mostra uma barra de mistura rica/ideal/pobre com o valor de λ, a rotação, o tempo de injeção, a correção do PI, o modo da malha e um gráfico de λ nos últimos 10 s. Pelo teclado, o visitante acelera o motor, aplica as perturbações de bico e de vazamento e alterna entre malha aberta e fechada.

Figura 4 - Painel do operador

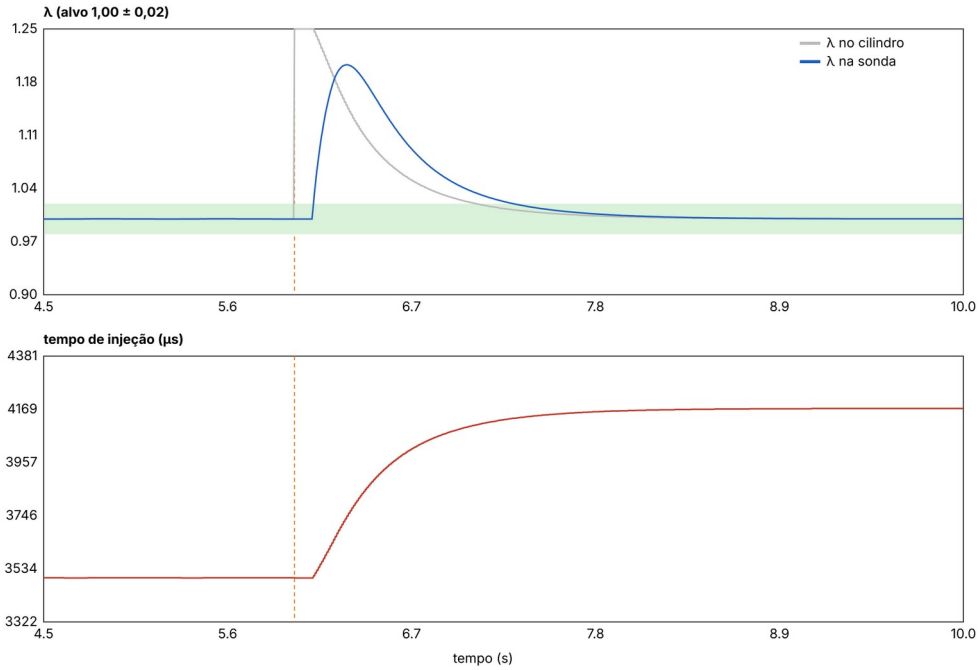


Fonte: Autoria própria (2026).

A verificação foi feita em três níveis, todos executados automaticamente a cada envio ao repositório. Nos testes unitários, o controle em C é testado com o framework Unity, e a planta, com testes Go. Na integração, o mesmo código C do controle roda dentro da planta Go (software-in-the-loop), e o firmware completo roda no Renode com casos em Robot Framework. No teste ponta a ponta, ECU, planta e testes conversam pelo vcan0. A Figura 5 mostra a resposta a uma redução de 20 % na vazão do bico a 2000 rpm: a mistura empobrece após o atraso do escapamento,

o PI eleva o tempo de injeção em cerca de 25 % e λ volta à faixa de ±0,02 em 1,3 s, sem sobressinal.

Figura 5 - Resposta a uma redução de 20 % na vazão do bico (2000 rpm, malha fechada)



Fonte: Autoria própria (2026).

5 CONSIDERAÇÕES FINAIS

A bancada atingiu as metas propostas: a tarefa de controle cumpriu o período de 10 ms com folga de mais de 98 % do processador e jitter dez vezes menor que o limite, e a malha corrigiu as perturbações em menos de 2 s. A separação entre amostragem por hardware, tarefa de controle acordada por notificação e comunicação em tarefa de menor prioridade manteve o caminho crítico curto e previsível. Como limitação, os tempos foram medidos no relógio virtual do emulador; a próxima etapa natural é gravar o mesmo binário em uma placa STM32 e repetir as medições com instrumentos reais.

REFERÊNCIAS

BARRY, R. *Mastering the FreeRTOS real time kernel*: a hands-on tutorial guide. [S. l.]: Real Time Engineers Ltd., 2016.

BOSCH. *Manual de tecnologia automotiva*. 25. ed. São Paulo: Blucher, 2005.

HEYWOOD, J. B. *Internal combustion engine fundamentals*. 2. ed. New York: McGraw-Hill, 2018.

KOPETZ, H. *Real-time systems*: design principles for distributed embedded applications. 2. ed. New York: Springer, 2011.

LIU, C. L.; LAYLAND, J. W. Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM*, v. 20, n. 1, p. 46-61, 1973.

OGATA, K. *Engenharia de controle moderno*. 5. ed. São Paulo: Pearson, 2010.

RENODE. *Renode documentation*. Antmicro. Disponível em: https://renode.readthedocs.io. Acesso em: 22 out. 2026.

STMICROELECTRONICS. *RM0091*: STM32F0x1/STM32F0x2/STM32F0x8 reference manual. Genebra: STMicroelectronics, 2017.

TANENBAUM, A. S.; BOS, H. *Sistemas operacionais modernos*. 4. ed. São Paulo: Pearson, 2016.